

XII: Les transformeurs

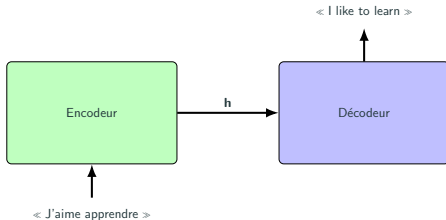
Guillaume Guex guillaume.guex@unil.ch

Les modèles seq2seq

Les modèles seq2seq

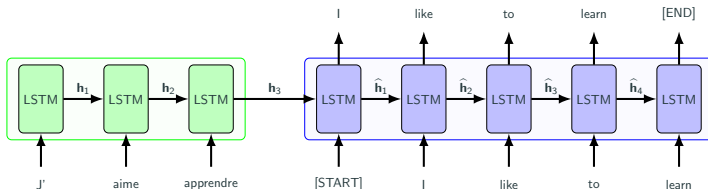
Les **modèles seq2seq** sont des modèles de ML qui prennent en entrée une séquence et retournent une séquence (pas nécessairement de taille identique). On peut voir leur tâche comme une **traduction**.

Généralement, les modèles seq2seq possèdent deux modules : un **encodeur** et un **décodeur**. Un **vecteur de contexte h** est transmis de l'encodeur au décodeur.



Les modèles seq2seq avec cellules récurrentes

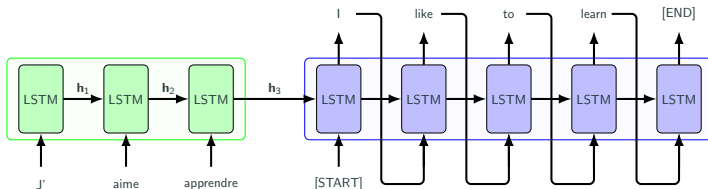
Les modèles seq2seq classiques utilisent des **cellules récurrentes**.
Le **dernier état caché** de la cellule d'encodage constitue le **premier état caché** de la cellule de décodage : c'est le vecteur de contexte.



L'**entraînement** consiste à prédire la phrase traduite décalée d'un token sur la droite.

Les modèles seq2seq : évaluation

Lors de l'**évaluation**, on va fournir au décodeur le token [START], et réinjecter le mot prédit dans la sortie suivante (ou faire un *beam search*).

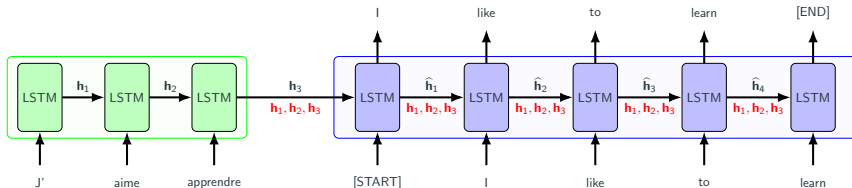


On s'arrête dès que le décodeur produit le token [END].

Les modèles seq2seq

Le problème avec ces modèles, c'est que les **vecteurs cachés s'enchaînent** et l'**information des premiers tokens de l'encodeur disparaît**.

Idéalement, on aimerait transférer **tous les états cachés de l'encodeur $h_1, \dots, h_e$ à chaque étape de décodage**.



Pour utiliser cette information le mécanisme d'**attention** est utilisé.

L'attention

L'attention

L'**attention** est un mécanisme de deep learning qui :

- Reçoit un vecteur d'entrée $\mathbf{q}$ appelé **requête (query)**.
- Interroge une série de vecteurs $\mathbf{k}_1, \dots, \mathbf{k}_p$ appelés **clés (keys)**, pour construire des **affinités** $a_j = a(\mathbf{q}, \mathbf{k}_j)$, avec $\sum_j a_j = 1$ et $a_j \geq 0$.
- En fonction de **valeurs (values)** $\mathbf{v}_1, \dots, \mathbf{v}_p$ associées aux clés, rend le **vecteur d'attention** suivant :

$$\text{Attention}(\mathbf{q}, \mathbf{k}_1, \dots, \mathbf{k}_p, \mathbf{v}_1, \dots, \mathbf{v}_p) = \sum_j a_j \mathbf{v}_j$$

Les affinités a_j construites peuvent s'interpréter comme l'**attention que porte $\mathbf{q}$ sur $\mathbf{k}_1, \dots, \mathbf{k}_p$** .

L'attention

Clés		2		6		1
		1		3		5
		4		2		7
Requête	4	↓		↓		↓
	5					
	6					
Affinités		0.7		0.2		0.1
Valeurs		.		.		.
		1		3		4
		2	+	5	+	4
		1		1		1
Attention				=		
				1.7		
				2.8		
				1.0		

Le vecteur d'affinités

Le **vecteur d'affinités** $\mathbf{a} = (a_j)$ entre une requête $\mathbf{q}$, de taille d , et les clés contenues dans une matrice $\mathbf{K} = (\mathbf{k}_1 | \dots | \mathbf{k}_p)^\top$ se calcule avec le **softmax des produits scalaires** (renormalisés) :

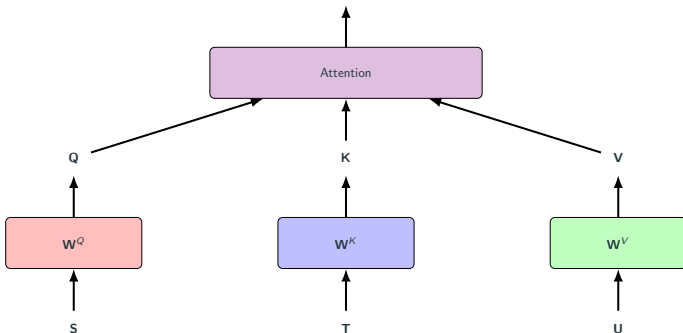
$$\mathbf{a} = \text{Softmax} \left(\frac{\mathbf{K}\mathbf{q}}{\sqrt{d}} \right).$$

Matriciellement, si l'on reçoit plusieurs requêtes contenues dans $\mathbf{Q} = (\mathbf{q}_1 | \dots | \mathbf{q}_l)^\top$, et que les valeurs sont contenues dans $\mathbf{V} = (\mathbf{v}_1 | \dots | \mathbf{v}_p)^\top$, on aura :

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Softmax} \left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}} \right) \mathbf{V}.$$

Les têtes d'attention

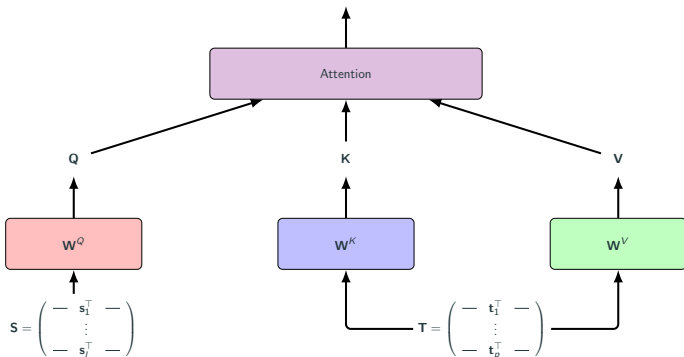
Dans un réseau de neurones, on peut créer des **têtes d'attention** qui possèdent trois couches entièrement connectées $\mathbf{W}^Q$, $\mathbf{W}^K$, $\mathbf{W}^V$, donnant respectivement les requêtes, les clés et les valeurs.



C'est l'**entraînement** qui va déterminer, pour chaque vecteur dans $\mathbf{S}$, où mettre de l'attention sur $\mathbf{T}$ pour sélectionner les valeurs transformées de $\mathbf{U}$.

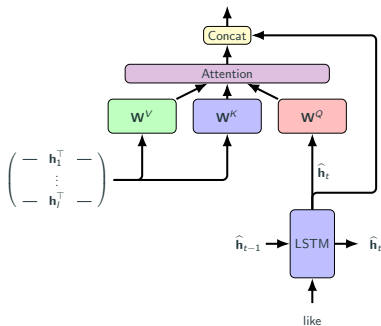
L'attention entre les séquences

Lorsque l'on a deux séquences de vecteurs $\mathbf{s}_1, \dots, \mathbf{s}_l$ et $\mathbf{t}_1, \dots, \mathbf{t}_p$, on peut utiliser l'attention pour apprendre à chaque mot d'une séquence à **porter son attention sur certains mots de l'autre séquence**.



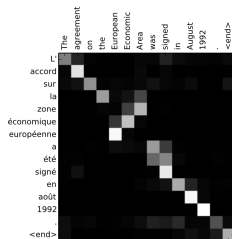
L'attention dans les modèles seq2seq récurrents

Dans un modèle seq2seq récurrent, on peut munir la cellule de décodage d'une tête d'attention, afin de **mettre de l'emphasis sur une partie des états cachés h_i de l'encodeur**.

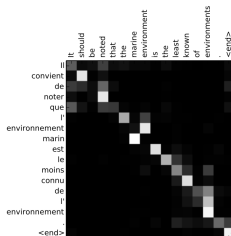


On concatène généralement la sortie de la cellule LSTM avec le résultat de l'attention.

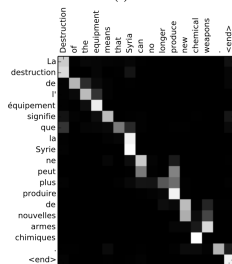
Les têtes d'attention entraînées



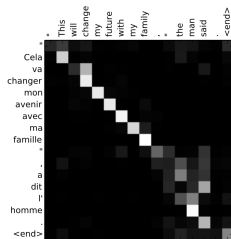
(a)



(b)



(c)



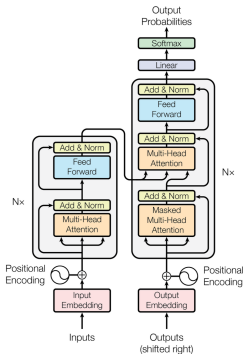
(d)

Source : D. Bahdanau et al. (2016) *Neural machine translation by jointly learning to align and translate*

Les transformeurs

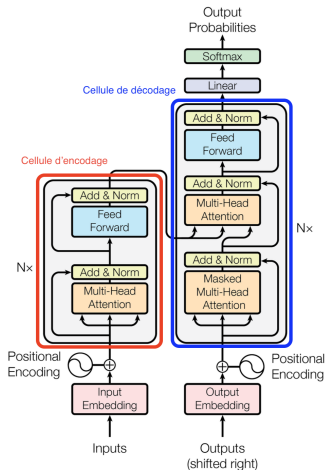
Les transformeurs

En 2017, dans l'article *Attention is all you need*, A. Vaswani et al. ont proposé un modèle seq2seq avec une architecture qui se base **uniquement sur le mécanisme d'attention** (plus de récurrence ou de convolutions). Ce modèle est nommé le **transformeur**.



Source : A. Vaswani et al. (2017) *Attention is all you need*

L'encodeur et le décodeur

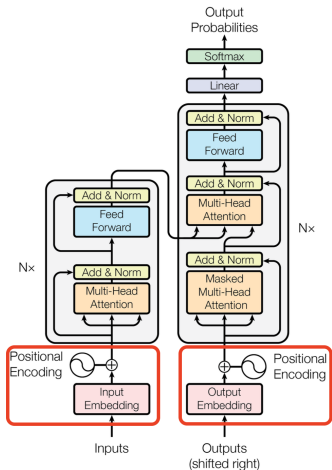


Le transformeur peut enchaîner plusieurs **cellules d'encodage**, qui vont finalement transmettre un contexte à plusieurs **cellules de décodage** empilées.

Les modèles de transformeur peuvent être plus ou moins lourds en fonction du **nombre de cellule d'encodage et de décodage**, de la **taille des vecteurs cachés** et du **nombre de têtes d'attention**.

Source : A. Vaswani et al. (2017) *Attention is all you need*

L'embedding avec encodage positionnel

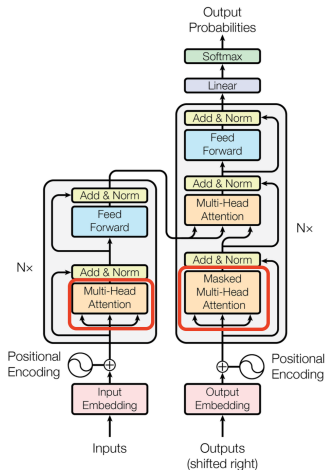


Le transformeur prend des séquences en entrées et en sortie (même principe d'entraînement que les modèles seq2seq classiques) jusqu'à une **taille maximale**.

Un **embedding** est effectué initialement sur tous tokens, et une information **positionnelle** est ajoutée aux composantes du vecteur, pour que le modèle puisse connaître la place des différents tokens dans la séquence.

Source : A. Vaswani et al. (2017) *Attention is all you need*

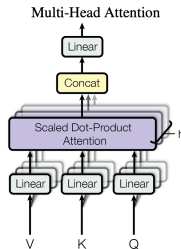
Les têtes de self-attention



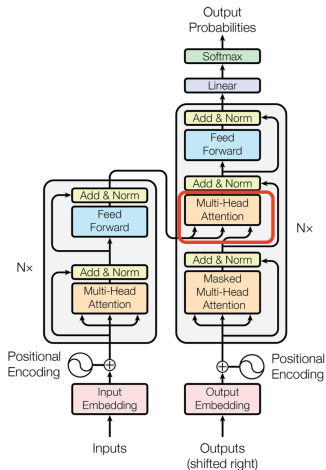
Source : A. Vaswani et al. (2017) *Attention is all you need*

Les cellules d'encodage et de décodage sont munies de têtes de **self-attention**, qui permettent d'apprendre les relations des différents tokens de la séquence entre eux. Dans le décodeur, les **tokens de droite sont masqués**.

Il s'agit de **têtes multiples**, permettant de construire plusieurs types de relations.



L'attention de décodeur sur l'encodeur

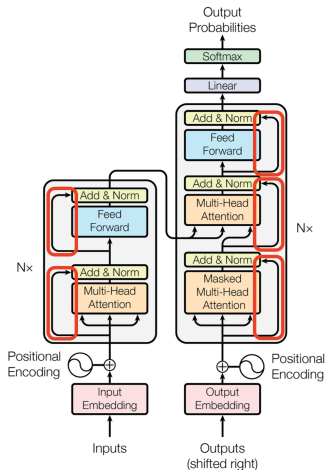


La sortie de toutes les couches d'encodage (un vecteur pour chaque token) est transmise à toutes les cellules de décodage via un mécanisme d'**attention** classique (similaire à celui du seq2seq récurrent).

Les tokens du décodeur vont **porter leur attention sur différentes parties de la séquence à traduire.**

Source : A. Vaswani et al. (2017) *Attention is all you need*

Les connexions résiduelles



Finalement, tous les modules des cellules d'encodage et de décodage sont munis de **connexions résiduelles**.

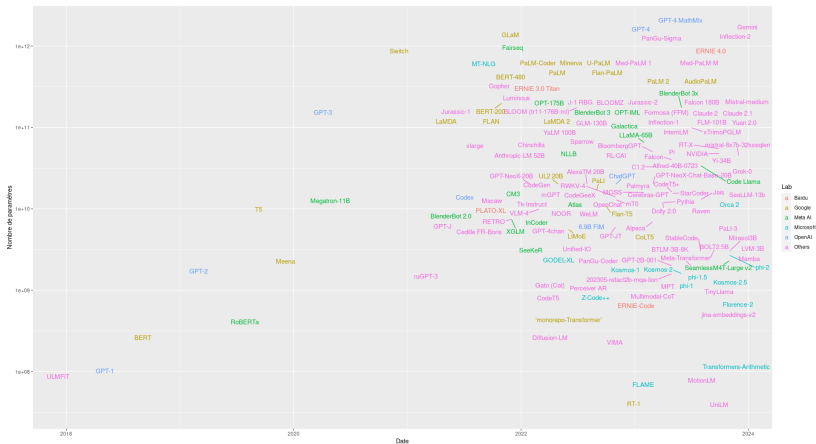
Ces dernières **permettent au gradient de se rétro-propager** plus facilement dans tout le réseau, et accélèrent l'entraînement.

Source : A. Vaswani et al. (2017) *Attention is all you need*

Les modèles de langage

Les modèles de langage pré-entraînés

Les transformeurs et le mécanisme d'attention ont été utilisés pour créer des **modèles de langage pré-entraînés**. Depuis 2018, on observe une explosion du nombre de modèles et de leur taille.



Les **caractéristiques de BERT (2018)** :

- Développé par **Google**.
- **Open source**.
- Existe en **plusieurs tailles**.
- Pour le grand modèle, **345 millions de paramètres**
- Entraîné avec **3.3 milliards de tokens**.
- Contexte de **512 tokens**.
- Entraîné sur les tâches de **prédiction d'un token masqué** et **prédiction de prochaine phrase** (2 phrases données, dire si le couple convient).
- Donne un **vecteur contextuel pour chaque token**, et un **vecteur global pour la séquence**.

Les **caractéristiques de GPT-4o (2024)** :

- Développé par **OpenAI**.
- **Propriétaire**.
- Existe en **2 tailles**.
- Pour le grand modèle, **200 milliards de paramètres**
- Entraîné avec **1000 milliards de tokens**.
- Contexte de **128k tokens**.
- Traite le texte, les images et le son.
- Entraîné sur la tâche de **prédiction du prochain mot**.
- Donne un **vecteur du contexte gauche pour chaque token**, et un **vecteur global pour la séquence**.

Fine-tuning des modèles de langage

Les modèles de langage peuvent être utilisés pour de nombreuses tâches grâce au **fine-tuning**.

1 - **Semi-supervised** training on large amounts of text (books, wikipedia..etc).

The model is trained on a certain task that enables it to grasp patterns in language. By the end of the training process, BERT has language-processing abilities capable of empowering many models we later need to build and train in a supervised way.

Semi-supervised Learning Step

Model:



Dataset:



Objective:

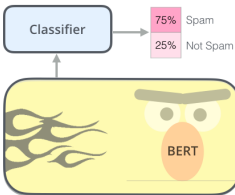
Predict the masked word
(language modeling)

2 - **Supervised** training on a specific task with a labeled dataset.

Supervised Learning Step

Model:

(pre-trained
in step #1)



Dataset:

Email message	Class
Buy these pills	Spam
Win cash prizes	Spam
Dear Mr. Atreides, please find attached...	Not Spam